

SimuFire: arquitectura actual y modelo de dos zonas

Fecha: 2026-06-15

Baseline de validacion CFAST: 14 fallos requeridos tras Phase 4B

1. Idea general

SimuFire es un motor Godot que combina una interfaz interactiva con un conjunto de subsistemas fisicos simplificados: combustion, temperatura, humo, O₂, CO/CO₂/HCN, ventilacion, presion, supresion, victimas y validacion contra referencias externas como CFAST.

El centro del sistema es `SimulationEngine.gd`. Ese nodo mantiene el tiempo de simulacion, contiene los parametros globales, crea los subsistemas y decide el orden en que se ejecuta cada paso fisico.

2. Arranque normal de la aplicacion

El arranque normal viene de `project.godot`:

```
project.godot
-> run/main_scene = res://scenes/MainMenu.tscn
-> escenas de UI / juego
-> Main.gd
-> World/SimulationEngine
```

`Main.gd` es la capa de aplicacion. En `_ready()` toma referencias a UI, camara, minimapa y al nodo `SimulationEngine`. En `_physics_process(delta)` actualiza interaccion/visualizacion y deja que el motor avance segun el modo de la escena.

La fisica no vive en `Main.gd`; vive en `sim/core` y `sim/fire`.

3. Arranque del motor

Cuando Godot instancia `SimulationEngine.gd`, se crean estos subsistemas:

<code>SimulationEngine</code>	
<code>CombustionSystem</code>	-> HRR, combustible, humo, CO, CO ₂ , HCN
<code>OxygenExchangeSystem</code>	-> consumo/intercambio de O ₂
<code>ThermalSystem</code>	-> temperaturas, capa caliente, energia
<code>GasExchangeSystem</code>	-> humo, CO/CO ₂ /HCN, presion, PPV, venteo
<code>HVACSystem</code>	-> impulsion/extraccion mecanica
<code>FireSpreadSystem</code>	-> propagacion e ignicion secundaria
<code>GlassFailureSystem</code>	-> rotura de ventanas
<code>ZoneFireSolver</code>	-> ledger two-zone de masa/energia/especies
<code>SimulationStateBuilder</code>	-> estado serializable para UI/validacion/log
<code>SimulationLogWriter</code>	-> log temporal de simulacion

En `_ready()`, `SimulationEngine`:

1. Resuelve el `BuildingModel` desde `building_path`.
2. Sincroniza parametros con `SmokeModel` y subsistemas auxiliares.
3. Resetea gases, O₂, HVAC y log.
4. Hace bootstrap de combustion sobre el edificio.

5. Enciende la sala inicial si `auto_ignite_on_ready=true`.
6. Escribe una muestra inicial de estado.

4. Bucle principal de simulacion

Cada paso llama `SimulationEngine.step(delta)`. El orden actual importa mucho porque algunos sistemas consumen lo que otros acaban de producir.

```
SimulationEngine.step(delta)
  1. Calcula dt y avanza sim_time_s
  2. Construye _opening_flow_cache para aperturas interiores
  3. Inicializa balance global de carbono si hace falta
  4. _step_pool_fires()
  5. _step_oxygen() antes del fuego si el modo de O2 lo requiere
  6. _step_fire()
  7. _step_co_oxidation()
  8. _step_targets()
  9. _step_oxygen() despues del fuego en modo legacy
 10. ThermalSystem.step()
 11. ZoneFireSolver.validate_conservation() si esta activo
 12. _step_suppression()
 13. _step_steam_decay()
 14. GlassFailureSystem.step()
 15. _step_gas_exchange()
 16. _step_hvac()
 17. _step_passive_fuel()
 18. FireSpreadSystem.step()
 19. ThermalSystem.reconcile_two_zone_building() si two-zone esta activo
 20. _clamp_rooms()
 21. Balance de carbono y guardrails de interfaz de capa
 22. Detectores, victimas, eventos, peaks y log
```

5. Que hace cada subsistema

CombustionSystem.gd

Calcula el fuego de cada sala: curva HRR, consumo de combustible, limitacion por O2, smoke yield, CO, CO2, HCN, smoldering, backdraft-like pool y efectos de ventilacion. Recibe un context construido por `SimulationEngine`, con parametros de O2, ventana, Kawagoe, temperatura, modo `fire_o2_mode`, etc.

OxygenExchangeSystem.gd

Actualiza O2 de sala y capas (`room.o2`, `o2_upper`, `o2_lower`). Gestiona consumo por fuego, infiltracion, intercambio por aberturas y casos especiales como `plume_lower_mode`. Tras Phase 4A ya no duplica el consumo de O2 en `plume_lower_mode`.

ThermalSystem.gd

Gestiona energia, temperaturas superior/inferior, altura de capa caliente, transferencia por aberturas, perdidas a paredes/ambiente, perfiles verticales y FED termica. Es el principal escritor del estado termico two-zone.

GasExchangeSystem.gd

Mueve humo y especies por presion, PPV, aberturas y ambiente. Tambien contiene la ODE de presion termodinamica de Phase 3. Tras el fix de Phase 3, las puertas interiores abiertas cuentan como area de alivio.

ZoneFireSolver.gd

No es "otro modelo de fuego" separado. Es el coordinador/ledger que intenta que `ThermalSystem`, `GasExchangeSystem` y `OxygenExchangeSystem` usen los mismos flujos de masa, energia y especies entre zonas. Su objetivo es evitar

contabilidad doble o inconsistencias entre subsistemas.

HVACSystem.gd

Aplica impulsión/extracción mecánica y transporta humo/especies por conductos o ventilación mecánica.

FireSpreadSystem.gd y GlassFailureSystem.gd

FireSpreadSystem decide igniciones secundarias. GlassFailureSystem abre cristales por temperatura o hazard probabilístico, y esos cambios entran luego en ventilación/gases.

SimulationStateBuilder.gd y SimulationLogWriter.gd

Construyen snapshots para UI, validación y logs. El validator lee estos datos para comparar contra CFAST.

6. Ruta de validación

La validación no arranca desde la UI. Usa scripts:

```
python scripts/simulation/validate_reference_cases.py
-> decide checks y lee reportes/logs
-> llama casos individuales cuando corresponde

sim/validation/run_case.ps1 -CaseName <caso>
-> localiza Godot console
-> godot --headless --path <repo> -- --validation-case=<caso>
-> escena Godot con CaseRunner.gd
-> CaseRunner carga sim/validation/cases/<caso>.json
-> CaseRunner configura BuildingModel + SimulationEngine
-> engine.reset_simulation()
-> loop fijo hasta duration_s
-> escribe sim/validation/reports/<caso>.json
```

CaseRunner.gd aplica engine_overrides, eventos de apertura, supresión y carbono. Al final calcula métricas y escribe JSON. validate_reference_cases.py compara esos resultados contra CSV de CFAST o contra contratos internos.

7. Que es un modelo de dos zonas

Un modelo de dos zonas representa cada habitacion como dos volumenenes bien mezclados:

```
Techo
  zona superior: gases calientes, humo, CO/CO2/HCN, menor O2
  interfaz: altura de capa caliente
  zona inferior: aire mas frio, respirable durante parte del incendio
Suelo
```

No resuelve cada punto del espacio como CFD. En lugar de millones de celdas, guarda unas pocas variables por sala:

```
upper_gas_kg, lower_gas_kg
upper_energy_kj, lower_energy_kj
temp_upper_c, temp_lower_c
o2_upper, o2_lower, room.o2
co2_kg, co_kg, hcn_kg, smoke_kg
thermal_layer_m
```

La idea fisica:

1. El fuego libera energia y productos.
2. La pluma caliente sube y alimenta la zona superior.
3. La zona superior crece hacia abajo: baja `thermal_layer_m`.
4. Las puertas/ventanas intercambian gas caliente por arriba y aire frio por abajo.
5. El O2 se consume donde el fuego realmente toma aire.
6. Las perdidas a paredes, ambiente y ventilacion sacan energia y masa.

CFAST tambien es un modelo zonal. La dificultad actual de SimuFire es que todavia hay partes legacy que mezclan variables room-average con variables upper/lower. Por eso algunos fallos restantes son arquitectonicos: comparar `room.o2` promedio contra UL02 de CFAST no es una comparacion perfecta.

8. Estado two-zone actual

El repo ya tiene `two_zone_solver_enabled=true` y `two_zone_opening_flow_enabled=true` por defecto. Eso significa que el motor usa una representacion de dos zonas para energia, capas y parte de los flujos.

Pero no todo es canonico aun:

- `ThermalSystem` ya proyecta masa/energia upper/lower.
- `ZoneFireSolver` coordina y valida flujos, pero algunas fases siguen siendo puente.
- `OxygenExchangeSystem` todavia conserva caminos legacy.
- `CombustionSystem` puede usar `fire_o2_mode=legacy`, `upper`, `lower` o `interface`, pero el acoplamiento completo fuego-zona aun no esta totalmente canonico.
- Algunas metricas reportan `room.o2` promedio mientras CFAST referencia UL02 de capa superior.

9. Como leer los fallos actuales

Tras Phase 4B quedan 14 fallos requeridos. La lectura tecnica es:

```
No todos los fallos significan "parametro mal calibrado".
Varios significan "el motor aun mezcla arquitectura legacy con two-zone".
```

Ejemplos:

- `r0_window_360`: Phase 4A arreglo el consumo duplicado de O2. Los fallos O2 restantes son brecha room-average vs upper-layer.
- `slow_growth_sealed`: Phase 4B mostro que temperatura y O2 no tienen rango de parametros compatible con `fire_o2_mode="upper"`.
- `corridor_chain`: candidato Phase 4C; mezcla transporte O2 inter-sala, temperatura y limitacion por O2.

10. Mapa mental rapido

```

UI / Juego
  Main.gd
    |
    v
SimulationEngine.gd
  |
  +-- CombustionSystem      produce HRR, humo, CO/CO2/HCN
  +-- OxygenExchangeSystem  consume/intercambia O2
  +-- ThermalSystem         actualiza capas y temperaturas
  +-- GasExchangeSystem     mueve humo/especies/presion/PPV
  +-- HVACSystem            ventilacion mecanica
  +-- FireSpreadSystem       igniciones secundarias
  +-- GlassFailureSystem     rotura de ventanas
  +-- ZoneFireSolver         consistencia two-zone
  |
  v
SimulationStateBuilder / LogWriter
  |
  +-- UI
  +-- logs
  +-- CaseRunner / validation reports

```

11. Glosario minimo

- HRR: heat release rate, potencia del fuego en kW.
- O2 upper/lower: oxigeno en zona superior/inferior.
- room.o2: lectura promedio o agregada de sala.
- thermal_layer_m: altura de la interfaz de humo/capa caliente.
- FED: dosis fraccional efectiva para incapacitation/toxicidad.
- FEC: concentracion efectiva fraccional de irritantes.
- PPV: ventilacion positiva.
- ACH: air changes per hour, infiltracion/renovacion.
- Kawagoe: limite de HRR por ventilacion exterior.